

Explanation of the Lean 4 (mathlib) Code for the Subset-Sum (SS) Condition

Big picture

You’re formalizing the “subset-sum support constraint” for transversal diagonal gates:

- A transversal diagonal gate assigns each computational basis string $s \in \{0,1\}^n$ a phase depending on a modular weighted sum

$$\sum_j a_j s_j \bmod m.$$

- For a logical basis state

$$|k_L\rangle = \sum_s c_s^{(k)} |s\rangle$$

to be an eigenvector with eigenvalue $e^{2\pi i b_k/m}$, every string s that appears with nonzero coefficient must satisfy

$$\sum_j a_j s_j \equiv b_k \pmod{m}.$$

- You formalize that as: **support** $\subseteq S_k$ (for finitary “support sets”) or as: **if amplitude is nonzero then weight** $= b_k$ (for functional amplitude vectors).

Then you prove a correctness lemma:

- If the SS condition holds, then the abstract diagonal action `Dact` acts as a **global phase** on the state.

This is the core “SS $\Rightarrow$ eigenstate under diagonal action” statement.

Imports and namespace

```
import Mathlib.Data.ZMod.Basic
import Mathlib.Data.Bool.Basic
import Mathlib.Data.Finset.Lattice.Basic
import Mathlib.Algebra.BigOperators.Fin

open scoped BigOperators
namespace SS
```

- `ZMod.Basic`: gives `ZMod m` and basic algebra on it.

- `Bool.Basic`: for `Bool` and simp lemmas about `if`.
- `Finset.Lattice.Basic`: gives coercions ($\uparrow \text{supp} : \text{Set } \alpha \rightarrow \text{Finset } \alpha$), subset relations, etc.
- `BigOperators.Fin`: provides $i : \text{Fin } n, \dots$ notation.

All of your definitions are placed in `namespace SS` so names don't pollute globally.

BitString

```
abbrev BitString (n : ℕ) : Type := Fin n → Bool
```

This is your encoding of a binary string of length n .

- A string $s \in \{0,1\}^n$ is a function $\text{Fin } n \rightarrow \text{Bool}$.
- This is convenient in Lean because $\text{Fin } n$ is the index type $\{0, \dots, n-1\}$.

Bitwise complement complement

The BD_{2m} symmetry constraint in the paper involves “bitwise complement” of strings. In Lean this is implemented as Boolean negation at each coordinate.

```
def complement {n : ℕ} (s : BitString n) : BitString n := fun i => !s i

@[simp] lemma complement_apply {n : ℕ} (s : BitString n) (i : Fin n) :
  complement s i = !s i := rfl

@[simp] lemma complement_complement {n : ℕ} (s : BitString n) :
  complement (complement s) = s := by
  funext i
  simp [complement]
```

So if $s \in \{0,1\}^n$, then $\bar{s}$ is defined by $(\bar{s})_i = 1 - s_i$. The lemma `complement_complement` formalizes that complement is an involution.

Section Basic

Variables

```
section Basic
variable {m n : ℕ} [NeZero m]
```

Here you initially assume $m \neq 0$ so you can treat $\text{ZMod } m$ as the usual modular ring. You later discovered some lemmas don't need it, so you used `omit` for those.

bit : $\text{Bool} \rightarrow \text{ZMod } m$

```
def bit (b : Bool) : ZMod m := if b then (1 : ZMod m) else 0
```

This is the map $\{0,1\} \rightarrow \mathbb{Z}_m$ that sends

- `false` $\mapsto 0$
- `true` $\mapsto 1$

This is the bridge between your Boolean bitstrings and arithmetic in $\text{ZMod } m$.

simp lemmas bit_false, bit_true

```
omit [NeZero m] in
@[simp] lemma bit_false : bit (m := m) false = 0 := by
  simp [bit]

omit [NeZero m] in
@[simp] lemma bit_true : bit (m := m) true = 1 := by
  simp [bit]
```

They just tell `simp` how to reduce `bit false` and `bit true`. You wrapped them in `omit [NeZero m]` because they don't actually use $m \neq 0$.

weight

```
def weight (a : Fin n → ZMod m) (s : BitString n) : ZMod m :=
  ∑ i : Fin n, a i * bit (m := m) (s i)
```

This is exactly the modular subset-sum:

- You provide an “angle vector” $a = (a_0, \dots, a_{n-1})$ as $\text{Fin } n \rightarrow \text{ZMod } m$.
- For a bitstring s , compute

$$w_a(s) = \sum_i a_i \cdot s_i \pmod{m},$$

where $s_i \in \{0, 1\}$ is implemented as `bit (s i)`.

This `weight` is the mathematical core of SS: it tells you what phase a diagonal gate assigns to $|s\rangle$.

Sk (support subset)

```
def Sk (a : Fin n → ZMod m) (bk : ZMod m) : Set (BitString n) :=
  { s | weight (m := m) a s = bk }
```

This is the SS subset

$$S_k = \{ s \in \{0, 1\}^n : w_a(s) = b_k \}.$$

In other words, `Sk a bk` is the set of computational basis strings that pick up the same phase label `bk`.

mem_Sk

```
omit [NeZero m] in
@[simp] lemma mem_Sk {m n : ℕ} (a : Fin n → ZMod m) (bk : ZMod m) (s : BitString n) :
  s ∈ Sk (m := m) a bk ↔ weight (m := m) a s = bk :=
  Iff.rfl
```

Just the definitional unfolding of membership in that set. Again it doesn't need `NeZero`.

SSConditionSupport

```
def SSConditionSupport (a : Fin n → ZMod m) (bk : ZMod m) (supp : Finset (BitString n)) : Prop :=  
  (↑supp : Set (BitString n)) ⊆ Sk (m := m) a bk
```

This is the SS condition stated as a support containment statement:

Every string in the (finite) support `supp` lies inside `Sk`.

This matches the paper’s statement $c_s^{(k)} \neq 0 \Rightarrow s \in S_k$, but phrased for a `Finset` support rather than amplitudes.

SSConditionSupport'

```
def SSConditionSupport' (a : Fin n → ZMod m) (bk : ZMod m) (supp : Finset (BitString n)) : Prop :=  
  ∀ { s } , s ∈ supp → weight (m := m) a s = bk
```

Same condition, but in “elementwise” form:

If $s \in \text{supp}$, then its weight equals `bk`.

SSConditionSupport_iff

```
omit [NeZero m] in  
theorem SSConditionSupport_iff  
  {m n : ℕ} (a : Fin n → ZMod m) (bk : ZMod m) (supp : Finset (BitString n)) :  
    SSConditionSupport (m := m) a bk supp ↔ SSConditionSupport' (m := m) a bk supp := by  
  constructor  
  • intro h s hs  
    have : s ∈ (↑supp : Set (BitString n)) := by simpa using hs  
    have : s ∈ Sk (m := m) a bk := h this  
    simpa [Sk] using this  
  • intro h s hs  
    have hs' : s ∈ supp := by simpa using hs  
    simpa [Sk] using h hs'
```

end Basic

This theorem proves those two formulations are logically equivalent:

$$\text{SSConditionSupport } a \text{ bk supp} \leftrightarrow \text{SSConditionSupport}' a \text{ bk supp}.$$

So you can use whichever is more convenient when proving things.

- The $\Rightarrow$ direction: from set inclusion you get the weight equation using `Sk`’s definition.
- The $\Leftarrow$ direction: from the pointwise property you build the set inclusion.

Again, you omit `[NeZero m]` because this proof doesn’t need $m \neq 0$.

Section BDSymmetry (BD_{2m} constraint)

This is the new piece you added to match the BD_{2m} (binary dihedral) symmetry constraint from the paper.

Physically, when the code has a transversal Pauli- X of the form $X^{\otimes n}$ acting as a logical X , the logical basis states satisfy

$$|1_L\rangle = X^{\otimes n}|0_L\rangle.$$

Applying X flips every physical bit, so at the level of basis strings and supports you expect a “complement symmetry”:

$$S_1 = \{\bar{s} : s \in S_0\}.$$

In the paper, BD_{2m} symmetry also enforces a constraint on the angle vector a :

$$\sum_i a_i \equiv -1 \pmod{m},$$

which ensures $0^n \in S_0$ and $1^n \in S_1$.

Angle constraint BDAngleCondition

```
section BDSymmetry

variable {m n : ℕ}

/-- BD_{2m} angle constraint: the total angle sum is '-1' in 'ZMod m'. -/
def BDAngleCondition (a : Fin n → ZMod m) : Prop :=
  ∑ i : Fin n, a i = -1
```

This is a direct translation of $\sum_i a_i \equiv -1 \pmod{m}$ into Lean. (It does not require the assumption [NeZero m].)

A helper simp lemma: bit_not

To relate weights of s and $\bar{s}$, you first prove that Boolean negation corresponds to $1 - \text{bit}$:

```
@[simp] lemma bit_not {m : ℕ} (b : Bool) :
  bit (m := m) (!b) = 1 - bit (m := m) b := by
  cases b <|> simp [bit]
```

Mathematically, this is just:

$$\text{bit}(1 - b) = 1 - \text{bit}(b).$$

Key identity: weight of a complemented string

```
theorem weight_complement (a : Fin n → ZMod m) (s : BitString n) :
  weight (m := m) a (complement s) =
    (∑ i : Fin n, a i) - weight (m := m) a s := by
  classical
  simp [weight, complement, bit_not, mul_sub, Finset.sum_sub_distrib]
```

This is the formal version of the computation

$$\sum_i a_i(1 - s_i) = \left(\sum_i a_i \right) - \sum_i a_i s_i.$$

Main lemma: complement_mem_S1_of_mem_S0

With the angle constraint and the SS definition of S_k , you prove:

```

theorem complement_mem_S1_of_mem_S0
  (a : Fin n → ZMod m) (hBD : BDAngleCondition (m := m) a)
  (s : BitString n) (hs : s ∈ Sk (m := m) a 0) :
  complement s ∈ Sk (m := m) a (-1) := by
simp [Sk] at hs
have hsum : (∑ i : Fin n, a i) = (-1 : ZMod m) := by
  simp [BDAngleCondition] using hBD
calc
  weight (m := m) a (complement s)
    = (∑ i : Fin n, a i) - weight (m := m) a s := weight_complement (m := m) a s
  _ = (-1 : ZMod m) - 0 := by simp [hsum, hs]
  _ = (-1 : ZMod m) := by simp

```

This is exactly the paper's reasoning: if $w_a(s) = 0$ and $\sum_i a_i = -1$, then

$$w_a(\bar{s}) = \left(\sum_i a_i \right) - w_a(s) = -1 - 0 = -1.$$

For the set equality later, you also prove the reverse direction `complement_mem_S0_of_mem_S1`, using that complement is an involution.

Supports are complement-related: Sk_complement_eq

```

theorem Sk_complement_eq (a : Fin n → ZMod m) (hBD : BDAngleCondition (m := m) a) :
  Sk (m := m) a (-1) = complement '' Sk (m := m) a 0 := by
ext s
constructor
• intro hs
  refine complement s, complement_mem_S0_of_mem_S1 (m := m) a hBD s hs, ?_
  simp
• rintro t, ht, rfl
  exact complement_mem_S1_of_mem_S0 (m := m) a hBD t ht

```

This says: under BD_{2m} symmetry, the phase-(-1) SS set is exactly the image of the phase-0 SS set under bitwise complement.

Section ResidueShiftScreen (Classical Distance Condition)

This section formalizes the **Residue-Shift Screen** from paper 2510.20728 (Equation 3). The screen condition ensures that no two codewords from different residue classes are Hamming-1 neighbors, which guarantees the classical distance $d(C) \geq 2$. This is crucial because it makes X-type and Y-type Knill-Laflamme conditions vanish automatically.

Bit flipping: flipBit

First, we define the operation of flipping a single bit at position i :

```

def flipBit (s : BitString n) (i : Fin n) : BitString n :=
  fun j => if j = i then !s j else s j

```

This creates a Hamming-1 neighbor of s by toggling the i -th bit. We also prove basic properties:

```

@[simp] lemma flipBit_apply_eq (s : BitString n) (i : Fin n) :
  flipBit s i i = !s i := by simp [flipBit]

@[simp] lemma flipBit_apply_ne (s : BitString n) (i j : Fin n) (hij : j ≠ i) :
  flipBit s i j = s j := by simp [flipBit, hij]

@[simp] lemma flipBit_flipBit (s : BitString n) (i : Fin n) :
  flipBit (flipBit s i) i = s

```

The last lemma shows that flipping the same bit twice returns the original string.

Key lemma: `weight_flipBit`

The central technical result relates the weight of a flipped string to the original:

```

theorem weight_flipBit (a : Fin n → ZMod m) (s : BitString n) (i : Fin n) :
  weight a (flipBit s i) = weight a s + (if s i then -a i else a i)

```

Mathematically, this says:

$$w_a(\text{flip}_i(s)) = w_a(s) + \begin{cases} -a_i & \text{if } s_i = 1 \\ +a_i & \text{if } s_i = 0 \end{cases}$$

The proof isolates the i -th term in the sum using `Finset.sum_eq_single_of_mem`:

- For $j \neq i$: the j -th term is unchanged since $(\text{flip}_i(s))_j = s_j$.
- For $j = i$: the term changes from $a_i \cdot s_i$ to $a_i \cdot (1 - s_i)$, giving a difference of $\pm a_i$.

We also provide a variant expressing this as a difference:

```

theorem weight_flipBit_sub (a : Fin n → ZMod m) (s : BitString n) (i : Fin n) :
  weight a (flipBit s i) - weight a s = if s i then -a i else a i

```

The screen condition: `ResidueShiftScreen`

```

def ResidueShiftScreen (a : Fin n → ZMod m) (S : Fin K → ZMod m) : Prop :=
  ∀ i : Fin n, ∀ j k : Fin K, j ≠ k →
    S j - S k ≠ a i ∧ S j - S k ≠ -a i

```

This is the formal statement of Equation (3) from the paper:

$$S_j - S_k \not\equiv \pm a_i \pmod{m} \quad \text{for all } i \text{ and } j \neq k.$$

Here $S : \text{Fin } K \rightarrow \text{ZMod } m$ assigns phase labels to the K logical basis states. The condition forbids the difference between any two distinct phase labels from equaling $\pm a_i$ for any weight component a_i .

Main theorem: `no_hamming1_neighbor_of_screen`

```

theorem no_hamming1_neighbor_of_screen
  (a : Fin n → ZMod m) (S : Fin K → ZMod m)
  (hScreen : ResidueShiftScreen a S)
  (hInj : Function.Injective S)
  {j k : Fin K} (hjk : j ≠ k)
  {s : BitString n} (hs : s ∈ Sk a (S j))
  {i : Fin n} :
  flipBit s i ∉ Sk a (S k)

```

This is the key result: *if the screen condition holds and $s \in S_j$, then flipping any bit of s cannot land in S_k for $k \neq j$.*

The proof proceeds by contradiction:

1. Assume $\text{flip}_i(s) \in S_k$.
2. From `weight_flipBit`: $w_a(\text{flip}_i(s)) = w_a(s) + (\pm a_i) = S_j + (\pm a_i)$.
3. But $\text{flip}_i(s) \in S_k$ means $w_a(\text{flip}_i(s)) = S_k$.
4. Therefore $S_k - S_j = \pm a_i$, contradicting the screen condition.

Corollaries

We provide two additional formulations:

```

theorem disjoint_hamming1_of_screen
  (a : Fin n → ZMod m) (S : Fin K → ZMod m)
  (hScreen : ResidueShiftScreen a S)
  (hInj : Function.Injective S) :
  ∀ j k : Fin K, j ≠ k → ∀ s ∈ Sk a (S j), ∀ i : Fin n, flipBit s i ∉ Sk a (S k)

```

This restates the main theorem in a more explicit “for all” form: the union of all supports has no Hamming-1 neighbors between different classes.

```

theorem hamming_distance_ge_2_of_screen
  (a : Fin n → ZMod m) (S : Fin K → ZMod m)
  (hScreen : ResidueShiftScreen a S)
  (hInj : Function.Injective S)
  {j k : Fin K} (hjk : j ≠ k)
  {s t : BitString n} (hs : s ∈ Sk a (S j)) (ht : t ∈ Sk a (S k)) :
  i : Fin n, t = flipBit s i

```

This is the distance formulation: any two codewords from different residue classes have Hamming distance ≥ 2 . In other words, no codeword in S_k can be obtained by flipping a single bit of any codeword in S_j (for $j \neq k$).

Physical significance

The Residue-Shift Screen is important because:

- It guarantees the classical code (union of all S_k) has minimum distance $d \geq 2$.
- This makes single-qubit X and Y errors always detectable.
- In the Knill-Laflamme framework, X-type and Y-type conditions become trivially satisfied when $d \geq 2$, since such errors cannot map between valid codewords.

- Only Z-type Knill-Laflamme conditions need to be checked separately (which involve amplitude constraints).

Section Disjointness

Setup

```
section Disjointness

variable {m n K : ℕ}          -- removed [NeZero m]
variable (a : Fin n → ZMod m)
variable (b : Fin K → ZMod m)
variable (supp : Fin K → Finset (BitString n))
```

Here you introduce:

- $b : \text{Fin } K \rightarrow \text{ZMod } m$ = the list of intended logical phase labels $(b_0, \dots, b_{K-1})$.
- $\text{supp } k$ = the finite support set for the k -th logical basis state.

Theorem supports_disjoint_of_injective

```
theorem supports_disjoint_of_injective
  (hb : Function.Injective b)
  (hSS : ∀ k, SSConditionSupport' (m := m) a (b k) (supp k)) :
  ∀ {k l : Fin K}, k ≠ l → Disjoint (supp k) (supp l) := by
  intro k l hkl
  classical
  refine Finset.disjoint_left.2 ?_
  intro s hsk hsl
  have hk : weight (m := m) a s = b k := hSS k hsk
  have hl : weight (m := m) a s = b l := hSS l hsl
  have hbl : b k = b l := by
    calc
      b k = weight (m := m) a s := hk.symm
      _ = b l := hl
  exact hkl (hb hbl)

end Disjointness
```

Meaning:

- If the phase labels $b \ k$ are all distinct (injective), and
- each support $\text{supp } k$ satisfies SS with its label $b \ k$,
- then supports for different logical indices cannot overlap.

This is the orthogonality-by-disjoint-support fact: if $b_k \neq b_\ell$, no basis string can appear in both $|k_L\rangle$ and $|\ell_L\rangle$ under the SS rule.

How the proof works (in plain math):

- Suppose a string s lies in both $\text{supp } k$ and $\text{supp } l$.
- Then SS says $\text{weight } a \ s = b \ k$ and also $\text{weight } a \ s = b \ l$.

- Hence $b_k = b_l$, contradicting injectivity unless $k = l$.
- Therefore they are disjoint.

This matches the statement in your manuscript that distinct b_k make S_k disjoint and thus (at least) support-level orthogonality.

Section DiagonalAction

This section switches from “supports as finsets” to “full amplitude vectors”.

Variables

```
section DiagonalAction

variable {m n : ℕ} -- no [NeZero m]
variable {α : Type*} [MonoidWithZero α]
```

- α is the coefficient type for amplitudes. You only assume `MonoidWithZero` because you need multiplication $(*)$ and a zero element.
- You deliberately don’t assume $\alpha = \mathbb{R}$ yet: this keeps it abstract and reusable.

State

```
abbrev State (n : ℕ) (k : Type*) : Type _ := BitString n → k
```

A “state” is an amplitude assignment $\psi : \{0,1\}^n \rightarrow \alpha$. So in physics notation: $\psi(s)$ is the coefficient of $|s\rangle$. (You fixed the universe issue by writing `Type _`.)

SSConditionState

```
def SSConditionState (a : Fin n → ZMod m) (bk : ZMod m) (α : State n α) : Prop :=
  ∀ s, α s ≠ 0 → weight (m := m) a s = bk
```

This is the most faithful Lean version of the manuscript SS condition: if amplitude at s is nonzero, then s must have the right subset-sum. It is literally the “support lies in S_k ”

```
def Dact (phase : ZMod m → α) (a : Fin n → ZMod m) (α : State n α) : State n α :=
  fun s => phase (weight (m := m) a s) * α s
```

This is an abstract diagonal operator acting on amplitude functions.

- In the computational basis, a diagonal gate acts by multiplying each basis vector $|s\rangle$ by some scalar depending on s .
- Here, the scalar is `phase (weight a s)`.

So this corresponds exactly to the transversal diagonal gate’s induced action

$$(D\psi)(s) = \omega^{w_a(s)}\psi(s),$$

but abstracted as `phase : ZMod m → α` instead of hardcoding ω^x .

Theorem Dact_eq_global_of_SS

```

theorem Dact_eq_global_of_SS
  (phase : ZMod m → ℤ) (a : Fin n → ZMod m) (bk : ZMod m) ( : State n ℤ)
  (hSS : SSConditionState (m := m) a bk ) :
  Dact (m := m) phase a = fun s => phase bk * s := by
classical
funext s
by_cases hs : s = 0
• simp [Dact, hs]
• have hw : weight (m := m) a s = bk := hSS s hs
  simp [Dact, hw]

end DiagonalAction

end SS

```

This is the “SS correctness” lemma: If ψ only has support on strings with weight $= b_k$, then applying the diagonal action multiplies every nonzero component by the same scalar **phase bk**. So the whole state is an eigenvector with eigenvalue **phase bk**.

How the proof works:

- Prove equality of functions, so **funext s**.
- Split into two cases:
 1. $\psi(s) = 0$: both sides are 0 after multiplication, so **simp** closes it.
 2. $\psi(s) \neq 0$: SS tells you **weight a s = bk**, so the phase is **phase bk**, and **simp** closes it.

This is the formal Lean statement that SS implies the state is an eigenstate of the diagonal gate.

What this file verifies (and what it doesn’t yet)

Verified / formalized

- The definition of subset-sum weights on bitstrings.
- The definition of the SS subsets S_k .
- The SS condition as “support subset” and as “nonzero amplitude $\Rightarrow$ correct congruence”.
- Bitwise complement on bitstrings (**complement**) and the involution property **complement_complement**.
- The BD_{2m} symmetry constraint on angles (**BDAngleCondition**) and its consequences, including **complement_mem_S1_of_mem_S0** and **Sk_complement_eq**.
- **Residue-Shift Screen** (Eq. 3 of paper 2510.20728):
 - Bit-flip operation **flipBit** and its properties.
 - Key lemma **weight_flipBit**: weight changes by $\pm a_i$ under bit flip.
 - Screen condition **ResidueShiftScreen**: $S_j - S_k \not\equiv \pm a_i \pmod{m}$.
 - Main theorem **no_hamming1_neighbor_of_screen**: screen $\Rightarrow$ no Hamming-1 neighbors between classes.

- Corollary `hamming_distance_ge_2_of_screen`: classical distance $d(C) \geq 2$.
- $SS \Rightarrow$ disjoint supports (under distinct phase labels).
- $SS \Rightarrow$ eigenstate under the abstract diagonal action (global phase).

Not yet formalized

- Z-type Knill-Laflamme conditions (linear constraints on amplitudes: $\sum_x (1 - 2x_i)p_{j,x} = t_i$).
- Concrete code verification (e.g., verifying specific codes from the catalogue).
- Control phases for multi-control gates.